AIORI-3 HACKATHON

PS-029

Application-Oriented Network Quality Evaluation Using QoO

Proposed Solution: QoO-X

Team: CoderScythe

Team Leader: Satyam Raj

Team Member: Pranaba Ballav

Faculty Mentor: Dr. Subhayu Ghosh

Team Code: to be added

Email: satyam.260103049@iiitbh.ac.in

Email: pranaba.260104042@iiitbh.ac.in

1. What the problem is really asking

Use the supplied QoO framework

We are not inventing a new QoO formula. The supplied implementation stays as our scoring engine.

Pick two application profiles

Two documented, mentor-approved profiles, with every parameter written down.

Stress the network

Six controlled scenarios covering delay, jitter and packet loss.

Measure more than QoO

Log QoO, latency, throughput and at least one metric the application itself reports.

Compare fairly

Both profiles go through exactly the same impairment conditions.

Explain the difference

Show how the choice of profile changes what 'good network quality' actually means.

2. Our solution: QoO-X



- One experiment ID ties together the profile, impairment, measurements and results.
- The same scenario is applied to both profiles, which keeps the comparison fair.
- Raw results are kept, so every dashboard number traces back to a real run.
- We add automation, reproducibility and interpretation. We do not touch the QoO mathematics.

3. Two application profiles

Profile A: WebRTC / Real-Time Video

Runs on the WebRTC application path.

Application-level observations: RTT, jitter, packet loss, bitrate, FPS, received resolution, freeze events and duration.

Why it matters: real-time apps are very sensitive to delay and jitter.

Profile B: DASH / Adaptive Streaming

Runs on the goDASH application path.

Observations: segment behaviour, throughput, QoO-related segment inputs and a playback metric.

Why it matters: a streaming client can react very differently to the same network conditions.

Important

The exact profile parameters will be confirmed with our faculty mentor before the first experiment is run.

4. Six experiments

ID	Delay	Jitter	Loss	Purpose
S0	0 ms	0 ms	0%	Baseline
S1	50 ms	0 ms	0%	Low delay
S2	100 ms	0 ms	0%	High delay
S3	0 ms	20 ms	0%	Moderate jitter
S4	0 ms	50 ms	0%	High jitter
S5	0 ms	0 ms	1%	Packet loss

Run rule

For every scenario:

1. reset the testbed
2. load the profile
3. apply the impairment
4. start the workload
5. collect metrics
6. save the raw data
7. repeat for the second profile

5. Measurement record

Identity

experiment_id
timestamp
profile_id
profile_version

Impairment

delay_ms
jitter_ms
packet_loss_percent

Network

latency_ms
throughput_mbps
measured_loss
jitter

QoO

qoo_score
QoO profile parameters
implementation version

Application

metric_name
metric_value
optional raw artifact path

Traceability

scenario_id
run number
notes /
error state

- Raw measurements are kept separate from averages.
- The exact configuration used is stored with every run.
- Missing measurements stay missing. We never fill gaps with made-up values.

6. A dashboard that tells the story

Overview

Current profile | scenario | QoO | latency | throughput | loss

Comparison

Profile A vs Profile B under the same scenario

Correlation

QoO against latency, jitter, loss and throughput

WebRTC KPIs

RTT | jitter | packet loss | bitrate | FPS | freezes

DASH KPIs

Segment behaviour | throughput | application metric

Experiment table

All six scenarios with run IDs and timestamps

The insight we want judges to see

We do not stop at a single number. We show how one network condition leads to different outcomes for different applications.

7. Technical implementation



- The container stack handles the emulated network path, impairment control and telemetry.
- A Python experiment controller manages profiles, scenarios, run IDs and exports.
- Analysis code turns raw measurements into comparison tables and plots.
- The repository holds configs, code, data schema, documentation and the commands to reproduce every run.

8. What makes our project different

Reproducible

Every run is identified, logged and can be replayed.

Profile-aware

Same network, different application requirements.

Application-linked

QoO is checked against metrics the application really reports.

Automated

No hand-built spreadsheets for six scenarios.

Traceable

Every dashboard value maps back to raw experiment data.

Extendable

New applications or scenarios can be added as config files.

9. Prototype demonstration

1. Start the testbed
2. Select Profile A
3. Run the baseline
4. Add 100 ms delay
5. Show QoO + app metric
6. Select Profile B
7. Repeat 100 ms delay
8. Show the comparison
9. Open the six-scenario dashboard

The winning moment

The evaluator watches the same 100 ms delay produce a different QoO and a different application experience for WebRTC and for DASH.

10. GitHub repository structure

Built so the evaluator can find everything quickly.

```
A3-PS029-<TEAMCODE>/
|-- Proposed-structure-hackathon.pdf
|-- README.md
|-- profiles/
|-- scenarios/
|-- controller/
|-- analysis/
|-- data/raw/
|-- data/processed/
|-- docs/
|    |-- methodology.md
|    '-- results.md
'-- video/
      '-- team-introduction-link.txt
```

- Meaningful commit messages: setup, scenario runner, metrics collection, documentation.
- A short README: what it does, architecture, how to run it, where results live.
- The video link goes in a plain-text file and in the README.
- No secrets, tokens or private credentials in GitHub.

11. Submission plan and dates

By 14 Oct 2026

Steps 1-7 done: GitHub collaboration, repository, ownership transfer, PDF, team video and video link.

16-31 Oct 2026

Mentoring and Steps 8-10: mentor meetings, full code and supporting files, working demo.

24-30 Nov 2026

Regional hackathon window, as per the current guideline.

Our six-day sprint (8-14 Oct)

Day 1: GitHub and roles | Day 2: proposal PDF | Day 3: video recording | Day 4: clone and run the testbed | Day 5: first experiment and dashboard skeleton | Day 6: cleanup and final submission check

Our rule

We will not mark any step as complete until it is visible and working in the repository.

12. Team roles and next steps

Team CoderScythe: two students and a faculty mentor.

Team Leader: Satyam Raj

Owns the testbed:

- Docker setup
- network scenarios
- experiment runner
- raw measurements

Team Member: Pranaba Ballav

Owns the evidence:

- dataset and charts
- dashboard
- result interpretation
- demo narration

Faculty Mentor: Dr. Subhayu Ghosh

Owns quality control:

- approves the profiles
- reviews every deliverable
- guides technical decisions
- helps with evaluator questions

Our immediate first move

Fork the AIORI GitHub template, rename it to our team code, add the AIORI account as a collaborator, and prepare the Step 5 PDF and the Step 6 video script.

13. BIBLIOGRAPHY

[1] QoO Testbed (IETF 126 Hackathon)

Supplied container-based QoO testbed: gateway, impairment control, InfluxDB, Grafana, goDASH and WebRTC workflows.

github.com/cj-ike-kunze/ietf-126-hackathon-qoo

[2] IETF 126 Hackathon: QoO project

IETF Quality of Outcome (QoO) activity and project description for the Hackathon.

wiki.ietf.org/en/meeting/126/hackathon

[3] AIORI-3 Online Hackathon Guidelines

Official rules, Level 1 steps (1-7), deadlines and submission requirements.

portal.aiori.in/aiori-3/hackathon-guideline/

[4] AIORI-3 Official Template Repository

Template repository forked for our team repo and its required structure.

github.com/aiori-hackathon/A3-PSXXX-TCXXX

[5] AIORI-3 Problem Statement PS-029

Application-Oriented Network Quality Evaluation Using QoO: scope, key requirements and deliverables.

portal.aiori.in/aiori-3/

[6] QoO-X Desktop v0.1.0 (our tool)

Team CoderScythe experiment controller built around the supplied QoO testbed; README and CHANGELOG in the repository.

qoo-x/ folder of the team repository

THANK YOU

Team CoderScythe | PS-029 | QoO-X

Satyam Raj (Team Leader) - Pranaba Ballav (Team Member) - Dr. Subhayu Ghosh (Faculty Mentor)

Questions and feedback are welcome